Workshop 6: Threads and Locking

Remember: Discuss each question with your parnter and write down a short answer. Writing is important to force you to really think about whether your analysis was right when you do the experiment later. *Downloads:* See the Workshop info on the class website.

Part 0 - What was confusing

Preparation

- Each person writes down what they wrote in the online reading summary for what was confusing or a question they had about the reading material, or something else they find confusing now.
- Write your selection here:

Group

- Go around the table and each person presents their point.
- Discuss which one the table finds most interesting.
- Write the table's selection here:
- Discuss how to explain it so it is not confusing, or answer it if it is a question, or what more information/help you would need to do so if you cannot.
- Write the table's answer here:

Presenting

- Choose someone from the group to present it to the class.
- Pesenter:
- Now wait for the TA to call on your group and have your presenter present.

For the rest of the questions you work at your own pace with your partner. If you need help or have questions, feel free to ask other groups or the TA. Don't forget to sign in for attendance before you leave.

Part 1 - Instruction interleaving (simulator)

Preparation

- Download the `x86.py` simulator for the book and make sure it runs.
- Open the book **Chapter 26** and go to the Homework section.

Question 0: What do the simulator flags do?

- Use the -help flag or the github README to answer the following questions:
- What does the -c flag do?
-
- What does the -M 2000 flag do?
-
- What does -i 1 do?
-
- What does -t 4 do?
-

- What does -R dx -a dx=3 do?
-

Question 1 from the book

Before you run the simulation: - What is the program loop.s doing? - - Do question 1.

Question 2 from the book

- If you run multiple threads of loop.s, will they share any memory?
-
- What does this imply about the possibility for data races?
-
- Do question 2.

Question 3 from the book (optional)

Question 4 from the book

- Look at the code fro looping-race-nolock.s. What is this code doing? (There are two parts: what does the top part do and what does the bottom part do? Note that it is using %bx instead of %dx.)
-
- Where are the values that are being incremented and decremented stored?
-
- Does this different location change the possibility of data races?
-
- Do question 4.
- Did you see a race?
-

Question 5 from the book

- Do question 5.
- Did you see a race?
-
- WHy did you not see a race?
-

Question 6 from the book

- Do question 6.
- What can you conclude about switching between the threads and when you do and do not get a data race?
-
- What is different about %bx and memory location 2000 with regards to races?
-

Question 7 from the book

- Do question 7.

- Make sure to print out the trace without the answer (e.g., not using -c) and figure out what should happen first.

Question 8 from the book (optional)

Questions 9 and 10 from the book

- Look at the code for wait-for-me.s. This code has the thread whose %ax is 1 will be the signaller and write 1 to memory location 2000. Threads with %ax set to zero will read from memory location 2000 until it turns to 1 and then end.
- Run the commands for questions 9 and 10.
- What does the change between the two of them change?
-
- Why is the output so different?
-
- Is this an efficient use of the CPU, and what should be changed to make it better?
-

Part 2 - Thread API

Preparation

- Download the code from the class website.

Overhead of threads vs. forking()

- Look at the `overhead_fork.c` and `overhead_thread.c` programs. These programs create processes (fork) and threads (thread) and time how long it takes to start them up.
- How much actual work does each forked process/thread do?
-
- Where will most of the time go?
-
- Run the overhead_fork.c and overhead_thread.c programs with 1, 100, and 1000.
- Compare the times. What can you say about the overhead of creating threads vs. processes?

	1 process/thread	100 process/thread	1000 process/thread
Processes			
Threads			

Unsynchronized data races

- Look a the `unsynchronized_parallel_count.c` program.
- What is the shared data that the threads are racing on?
-
- Where is this shared variable stored?
-
- What part of the code has to overlap for there to be a race? (E.g., what is the critical section?)
-

- How many lines of code is this section?
-
- How can this interleave if it is so few lines of code?
-
- Run the program with 1, 2, 4, and 16 threads for 10 increments.
- Run the program with 1, 2, 4, and 16 threads for 1M increments.

count (10)	1 Thread	2 Threads	4 Threads	16 Threads
Data Races				
Performance				

count (1M)	1 Thread	2 Threads	4 Threads	16 Threads
Data Races				
Performance				

- What do you observe about data races with the number of threads and increments per thread? Why does the number of increments per thread affect the likelihood of data races?
-
- How does the performance change with the number of threads and number of increments? (Think about how many CPUs your machine has and what you saw in the fork vs. thread question.)
-

Implementing a lock without atomics

- Look at the `broken_synchronization_parallel_count.c` program. How is this program trying to avoid the data race?
-
- Why isn't this going to work?
-
- Run the program with 1, 2, 4, and 16 threads for 10 increments.
- Run the program with 1, 2, 4, and 16 threads for 1M increments.

count (10)	1 Thread	2 Threads	4 Threads	16 Threads
Data Races				
Performance				

count (1M)	1 Thread	2 Threads	4 Threads	16 Threads
Data Races				
Performance				

- How do the data races differ from the `unsynchronized_parallel_count.c` example?
-
- How has the performance changed? (And why?)
-

Using a real lock

- Look at the `correctly_synchronized_parallel_count.c` program. How is this program trying to avoid the data race?
-
- Why is this going to work?
-
- Run the program with 1, 2, 4, and 16 threads for 10 increments.
- Run the program with 1, 2, 4, and 16 threads for 1M increments.

count (10)	1 Thread	2 Threads	4 Threads	16 Threads
Data Races				
Performance				

count (1M)	1 Thread	2 Threads	4 Threads	16 Threads
Data Races				
Performance				

- How do the data races differ from the previous examples?
-
- How has the performance changed? (And why?)
-

A faster locking approach

- Look at the `faster_synchronized_parallel_count.c` program. What is this program doing differently?
-
- Why would this be faster? (How many times will the lock be used if you have 16 threads?)
-
- Run the program with 1, 2, 4, and 16 threads for 10 increments.
- Run the program with 1, 2, 4, and 16 threads for 1M increments.

count (10)	1 Thread	2 Threads	4 Threads	16 Threads
Data Races				
Performance				

count (1M)	1 Thread	2 Threads	4 Threads	16 Threads
Data Races				
Performance				

- How do the data races differ from the previous examples?
-
- How has the performance changed? (And why?)
-
- What is the tradeoff in accuracy vs. performance here?
- The count value is only correct when a thread finishes. While threads are running it doesn't update.

You're done for the day! *Don't forget to sign the attendance sheet.*